



Contents

- 1. Mindset & Ground Rules**
- 2. What Is Njirani — Define Your North Star**
- 3. The 9-Month Arc at a Glance**
- 4. Season 1: The Foundation (Months 1–3)**
 - Month 1 — Git, Linux, Schema Design, First API
 - Month 2 — Testing, Docker, CI/CD, First Deployment
 - Month 3 — API Craft, M-Pesa, Email/SMS, Background Jobs
- 5. Season 2: The Integrations (Months 4–6)**
 - Month 4 — Database Mastery
 - Month 5 — Caching & Performance
 - Month 6 — Security & Observability
- 6. Season 3: The Polish (Months 7–9)**
 - Month 7 — Next.js Frontend
 - Month 8 — Flutter Mobile Client
 - Month 9 — AI Feature & Infrastructure as Code
- 7. Your Daily Ritual**
- 8. Tools & Platforms**
- 9. Books Worth Buying**
- 10. The One Metric That Matters**

1. Mindset & Ground Rules

This is a marathon, not a sprint. Every day you show up and build or add one feature — no matter how small. The journey matters more than the destination. Nine months from now you will proudly call yourself a backend engineer because you earned it.

Non-negotiable rules — read these every week:

- **Ship ugly, refine later.** A working messy endpoint beats a perfect one in your head.
- **One concept per week.** Go deep on one thing. Don't jump around topics.
- **If you're not stuck, you're not learning.** Confusion is the signal that growth is happening. Sit with it.
- **Read the docs, not just tutorials.** Tutorials show the happy path. Docs show everything.
- **Commit every day — even one line.** Green squares on GitHub are discipline made visible.
- **Write before you code.** Before each feature, write in plain English what it should do, what can go wrong, and how you'll test it.
- **Progress over perfection.** A pushed commit beats a perfect local branch.
- **Every four weeks, take a lighter review week.** Consolidate what you've learned before accelerating.

2. What Is Njirani — Define Your North Star

Before writing a single line of code, spend one full day answering this question:

What real problem in my community does Njirani solve?

Ideas to spark thinking:

- Community savings group (chama) management + M-Pesa contributions
- Rental property management + automated rent collection via M-Pesa
- Local service marketplace connecting fundis and clients
- School fees management system for small private schools
- Bodaboda fleet earnings and maintenance tracker

What to do on Day 1:

- Pick the idea that genuinely interests you — you'll live with this for 9 months.
- Write a one-paragraph problem statement and pin it somewhere visible.
- Every feature you build for the next 9 months should answer to that paragraph.
- Create the GitHub repo — name it 'njirani'. Public. No README yet.
- Clone it locally. Build the monorepo skeleton: /backend, /frontend, /mobile, /docs.

3. The 9-Month Arc at a Glance

Month	Focus	Key deliverable
1–3	Season 1: Foundation	Monorepo · PostgreSQL schema · FastAPI API · Auth · Tests · Docker
4–6	Season 2: Integrations	M-Pesa Daraja · SMS (Africa's Talking) · Email · Celery background job
7–9	Season 3: Polish	Next.js frontend · Flutter mobile · Claude AI feature · Terraform IaC · G

Three seasons. Each one builds on the last. Don't jump to Season 2 until your Season 1 foundation is solid — a shaky base collapses under real integrations.

4. Season 1: The Foundation (Months 1–3)

Month 1 — Git, Linux, Schema Design, First API

What you're learning: Professional dev workflow, database design, your first working FastAPI app. What you're building: The skeleton — repo, project structure, database schema, 2–3 working endpoints.

Week 1 — Git Mastery + Linux Fluency

Focus: The non-negotiable engineering bedrock

Daily tasks:

- **Day 1:** Set up the monorepo. Understand every file you create and why.
- **Day 2:** Learn Git branching. Create a develop branch. All work happens on feature branches, never on main.
- **Day 3:** Linux terminal deep dive — file permissions, processes, grep, curl, htop, SSH basics.
- **Day 4:** Write your .gitignore, README.md, and docker-compose.yml for Njirani. Make them real.
- **Day 5:** Learn conventional commits: feat:, fix:, chore:, docs:. Every commit from today follows this.

Resources:

- learngitbranching.js.org — best interactive Git tutorial. Do every level.
- linuxjourney.com — covers exactly the Linux a backend dev needs.
- conventionalcommits.org — the commit standard used by professional teams.
- ohshitgit.com — bookmark this. You will need it.

Week 2 — Database Schema Design

Focus: Design on paper before you touch code

Daily tasks:

- **Day 1:** Study relational database design — primary keys, foreign keys, normalisation (1NF, 2NF, 3NF).
- **Day 2:** Draw your Njirani schema on paper. Every entity, every relationship. No computer yet.
- **Day 3:** Learn what EXPLAIN ANALYZE does in PostgreSQL. Understand indexes and when to use them.
- **Day 4:** Translate your paper schema into Alembic migrations. Run them. See your tables appear.
- **Day 5:** Seed the database with realistic Kenyan test data — real names, M-Pesa-style phone numbers.

Resources:

- postgresql.org/docs/current/tutorial.html — the official PostgreSQL tutorial.
- drawsql.app — free tool for drawing schemas visually before you code.
- alembic.sqlalchemy.org — specifically the tutorial section.
- use-the-index-luke.com — the best free resource on database indexing. Bookmark it.

Week 3 — Your First Real FastAPI Endpoints

Focus: Connect your API to a real database

Daily tasks:

- **Day 1:** Read the entire FastAPI tutorial at fastapi.tiangolo.com/tutorial. All of it. Take notes.
- **Day 2:** Build your first router — whatever your core resource is (users, properties, groups, etc.).
- **Day 3:** Connect your router to a real PostgreSQL database using SQLAlchemy ORM.
- **Day 4:** Understand Pydantic schemas — the difference between input, output, and database models.
- **Day 5:** Test every endpoint manually using Swagger UI at /docs. Fix everything that breaks.

Resources:

- fastapi.tiangolo.com/tutorial — treat it as your textbook this week.
- sqlalchemy.org/en/20/orm/quickstart.html — SQLAlchemy 2.0 ORM quickstart.
- testdriven.io/blog/fastapi-sqlalchemy — FastAPI + SQLAlchemy in a real project.

Week 4 — Authentication Done Right

Focus: JWT, bcrypt, refresh tokens, role-based access

Daily tasks:

- **Day 1:** Understand how JWT works — header, payload, signature. Draw the auth flow on paper.
- **Day 2:** Implement registration with bcrypt password hashing.
- **Day 3:** Implement login — return access token + refresh token.
- **Day 4:** Implement protected routes using FastAPI's Depends() system.
- **Day 5:** Implement role-based access control. Some routes are admin-only.

Resources:

- jwt.io — paste any JWT and decode it. Understand the structure.
- fastapi.tiangolo.com/tutorial/security — the OAuth2 with JWT section specifically.
- cheatsheetseries.owasp.org — OWASP Authentication Cheat Sheet. Read to understand risks.
- passlib.readthedocs.io — the library you'll use for hashing.

Month 2 — Testing, Docker, CI/CD, First Deployment

What you're learning: Writing real tests, containerisation, getting your API live on the internet. What you're building: A tested, containerised, automatically-deployed API with a public URL.

Week 5 — pytest and Real Test Coverage

Focus: Tests are the proof that your code works

Daily tasks:

- **Day 1:** Read the pytest docs getting started guide. Write your first 10 tests for existing code.
- **Day 2:** Learn fixtures and conftest.py. Refactor your tests to use them.
- **Day 3:** Learn pytest-cov. Get a coverage report. Identify untested code.
- **Day 4:** Write integration tests for every API endpoint using TestClient.
- **Day 5:** Target 70% coverage minimum. Write tests until you hit it.

Resources:

- docs.pytest.org — the official pytest docs. Your primary resource.
- realpython.com/pytest-python — the best single article on pytest.
- testdriven.io/blog/fastapi-crud — testing FastAPI CRUD apps specifically.
- Brian Okken's 'Python Testing with pytest' — the book to buy if you can.

Week 6 — Docker

Focus: Containerise everything so it runs the same everywhere

Daily tasks:

- **Day 1:** Understand containers conceptually. What problem does Docker actually solve?
- **Day 2:** Write a Dockerfile for your FastAPI backend. Build it. Run it.
- **Day 3:** Write a docker-compose.yml that runs API + PostgreSQL + Redis together.
- **Day 4:** Learn multi-stage builds — make your image smaller and production-ready.
- **Day 5:** Make docker-compose up start your entire backend with one command. No manual steps.

Resources:

- docs.docker.com/get-started — Docker's own getting started guide.
- testdriven.io/blog/dockerizing-fastapi — FastAPI + Docker specifically.
- docs.docker.com/compose — Docker Compose documentation.
- labs.play-with-docker.com — free browser-based Docker environment.

Week 7 — GitHub Actions CI/CD

Focus: Automate testing and deployment forever

Daily tasks:

- **Day 1:** Read the GitHub Actions documentation overview. Understand workflows, jobs, steps, triggers.
- **Day 2:** Write a workflow that runs your pytest suite on every push.
- **Day 3:** Add linting (ruff) to your CI pipeline.
- **Day 4:** Add a step that builds your Docker image and confirms it builds.
- **Day 5:** Set up branch protection rules — nothing merges to main if tests are failing.

Resources:

- docs.github.com/en/actions — the official Actions documentation.
- github.com/nektos/act — run GitHub Actions locally before pushing.
- [ruff.rs](https://github.com/charliermarsh/ruff) — the fastest Python linter. Use it instead of flake8.

Week 8 — First Cloud Deployment (GCP Cloud Run)

Focus: Get your API live on the internet with a real URL

Daily tasks:

- **Day 1:** Create a GCP account. Set up the gcloud CLI. Understand the free tier limits.
- **Day 2:** Push your Docker image to Google Artifact Registry.
- **Day 3:** Deploy to Cloud Run manually from the CLI. Get your first public URL.
- **Day 4:** Connect a managed PostgreSQL database — use Neon or Supabase on the free tier.
- **Day 5:** Wire GitHub Actions to auto-deploy to Cloud Run on every push to main.

Resources:

- cloud.google.com/run/docs/quickstarts — Cloud Run quickstart guides.
- neon.tech/docs — Neon serverless Postgres. Free tier is generous.
- cloud.google.com/artifact-registry/docs — pushing Docker images to GCP.
- console.cloud.google.com/billing/budgets — set a budget alert on day one.

Month 3 — API Craft, M-Pesa, Email/SMS, Background Jobs

What you're learning: Building APIs the right way, integrating real Kenyan payment infrastructure, sending real notifications, processing events asynchronously.

Week 9 — API Design Best Practices

Focus: Build APIs that are a pleasure to consume

Daily tasks:

- **Day 1:** Read about REST API design principles. Study pagination, filtering, sorting patterns.
- **Day 2:** Implement proper pagination (limit/offset or cursor-based) on all list endpoints.
- **Day 3:** Implement filtering and search on your main resources.
- **Day 4:** Implement API versioning (/api/v1/). Understand why this matters for production.
- **Day 5:** Implement consistent error responses — every error returns the same JSON structure.

Resources:

- restfulapi.net — comprehensive REST API design reference.
- jsonapi.org — the JSON API spec. Reading it teaches you a lot about design decisions.
- Microsoft REST API guidelines on GitHub — real-world design decisions explained.
- stoplight.io/api-design-guide — practical API design guide.

Week 10 — M-Pesa Daraja API

Focus: The hardest week — push through. This is where you earn it.

Daily tasks:

- **Day 1:** Create a Safaricom developer account at developer.safaricom.co.ke. Read all Daraja docs first.
- **Day 2:** Understand the STK Push flow completely on paper before touching code. Draw the sequence.
- **Day 3:** Use Cloudflare Tunnel to expose your localhost so Daraja can hit your callback URL.
- **Day 4:** Implement the STK Push endpoint. Test it with the Daraja sandbox.
- **Day 5:** Implement the callback handler that receives payment confirmation. Store it in the database.

Resources:

- developer.safaricom.co.ke/docs — the official Daraja documentation. Your bible this week.
- cloudflare.com/products/tunnel — Cloudflare Tunnel for exposing localhost. Free.
- YouTube: search 'Daraja API Python tutorial' — watching someone else struggle first helps.

Daraja will humble you. You'll hit errors, wrestle with callbacks, and question everything. This is normal. Every backend engineer in Kenya has been exactly here. Push through it — wiring M-Pesa is a skill that makes you immediately hireable.

Week 11 — Email and SMS

Focus: Real notifications for real users

Daily tasks:

- **Day 1:** Understand SMTP and transactional email. Set up Gmail SMTP or SendGrid free tier.
- **Day 2:** Implement email sending with HTML templates. Send a welcome email on registration.
- **Day 3:** Register at africastalking.com. Read their SMS API documentation.
- **Day 4:** Implement SMS sending via Africa's Talking. Send confirmation on key events.
- **Day 5:** Make both configurable via environment variables so you can swap providers later.

Resources:

- africastalking.com/docs — Africa's Talking API docs. Their support is excellent.
- sendgrid.com/docs — SendGrid documentation. 100 free emails/day.
- jinja.palletsprojects.com — Jinja2 for HTML email templates.
- python-dotenv.readthedocs.io — managing environment variables properly.

Week 12 — Background Jobs (Celery + Redis)

Focus: Never block an HTTP response on slow work

Daily tasks:

- **Day 1:** Understand why background jobs exist. What breaks if you process a callback synchronously?
- **Day 2:** Set up Redis locally and in docker-compose. Understand what a message broker does.
- **Day 3:** Set up Celery in your FastAPI project. Write your first task.
- **Day 4:** Move email sending and SMS to background tasks — they should never block a response.
- **Day 5:** Implement a scheduled task with Celery Beat — something that runs on a schedule.

Resources:

- docs.celeryq.dev — official Celery documentation.
- redis.io/docs — Redis docs. Read 'Introduction' and 'Data types' sections.
- testdriven.io/blog/fastapi-and-celery — FastAPI + Celery integration specifically.
- flower.readthedocs.io — Flower: a web UI to monitor Celery tasks. Useful for debugging.

5. Season 2: The Integrations (Months 4–6)

Your foundation is solid. This season is about going deeper and making Njirani genuinely useful — database mastery, caching, security, and observability.

Month 4 — Database Mastery

What to study and build:

- Run EXPLAIN ANALYZE on your slowest queries. Fix at least one slow query this month.
- Add database indexes to your most queried columns. Measure the before and after.
- Learn connection pooling — understand why you can't open a new DB connection on every request. Set up PgBouncer or use SQLAlchemy's built-in pooling.
- Implement soft deletes — records that are 'deleted' but not actually removed from the database.
- Learn database transactions properly — when to use them, what isolation levels mean.

Resources:

- use-the-index-luke.com — the whole site. Required reading this month.
- pgexercises.com — interactive PostgreSQL exercises. Do these every morning like LeetCode.
- pganalyze.com/docs/explain — understanding EXPLAIN ANALYZE output.
- postgresql.org/docs/current/mvcc.html — transactions and concurrency in PostgreSQL.

Month 5 — Caching & Performance

What to study and build:

- Implement Redis caching on your most frequently read endpoints.
- Learn cache invalidation — when should the cache be cleared? This is harder than it sounds.
- Implement rate limiting on your API endpoints.
- Learn what a CDN does and set one up via Cloudflare for your static assets.
- Run your first load test and understand what happens to your API under pressure.

Resources:

- redis.io/docs/manual/client-side-caching — Redis caching patterns.
- locust.io — Python-based load testing. Use it to stress test your API.
- fastapi-limiter.readthedocs.io — rate limiting for FastAPI.
- Martin Kleppmann's 'Designing Data-Intensive Applications' — start reading this month. The most important backend book that exists.

Month 6 — Security & Observability

What to study and build:

- Go through the OWASP Top 10 list and check your API against every single item.
- Implement structured logging — every log entry should be JSON with consistent fields.

- Set up Sentry for error tracking. Know when production breaks before your users do.
- Implement a /health and /metrics endpoint. Add Prometheus metrics.
- Audit your dependencies for vulnerabilities using pip audit.

Resources:

- owasp.org/Top10 — the OWASP Top 10. Read every item.
- sentry.io/for/python — Sentry Python SDK documentation.
- python-json-logger.readthedocs.io — structured JSON logging for Python.
- prometheus.io/docs/introduction/overview — Prometheus concepts.
- github.com/pypa/pip-audit — scanning Python dependencies for vulnerabilities.

6. Season 3: The Polish (Months 7–9)

The hardest technical work is done. Now you make Njirani look and feel like a real product — frontend, mobile, AI, and infrastructure as code.

Month 7 — Next.js Frontend

What to study and build:

- Build the web client that consumes your Njirani API.
- Learn Next.js App Router properly — server components vs client components.
- Handle JWT authentication on the frontend — storing tokens safely.
- Build at least three fully functional pages connected to real API data.

Resources:

- nextjs.org/docs — official Next.js documentation.
- nextjs.org/learn — the official interactive Next.js course. Do this first.
- tanstack.com/query — React Query for data fetching and caching on the frontend.
- ui.shadcn.com — component library built on Tailwind. Makes things look good fast.

Month 8 — Flutter Mobile Client

What to study and build:

- Build the mobile client that hits the same API as your web frontend.
- Learn Flutter's widget system and state management basics.
- Implement the core user flows on mobile — login, dashboard, key actions.
- Get it running on your own phone.

Resources:

- flutter.dev/docs/get-started — official Flutter getting started guide.
- flutter.dev/docs/cookbook — Flutter cookbook. Short practical recipes.
- riverpod.dev — the state management library to use in Flutter.
- pub.dev — Flutter package repository. The npm of the Flutter world.

Month 9 — AI Feature & Infrastructure as Code

What to study and build:

- Add one Claude API-powered feature to Njirani — natural language search, smart assistant, or auto-categorisation.
- Learn Terraform basics. Define your entire GCP infrastructure as code.
- Set up GitOps — push to GitHub, everything deploys automatically, no manual steps.
- Write proper documentation — API docs, architecture diagram, README that tells the whole story.

Resources:

- docs.anthropic.com — the Claude API documentation. Start with the Messages API.
- developer.hashicorp.com/terraform — official Terraform documentation.
- gruntwork.io/guides — practical Terraform tutorials.
- c4model.com — the C4 model for drawing architecture diagrams. Use for your docs.

7. Your Daily Ritual (Non-Negotiable)

Every single day, before you open your code editor, follow these four steps. This habit is the difference between someone who codes and someone who grows.

1 Write first (5 min)
Open a notebook or Notion page. Write: What am I building today? What concept am I learning? What could go wrong?

2 Code (your available time)
One focused session. One feature or concept. No tab switching. If you're stuck for more than 30 minutes, read the docs — not tutorials.

3 Commit (5 min)
Even if it's broken, commit with a note explaining where you stopped. Use conventional commits: feat:, fix:, chore:. Future you will thank you.

4 Log it (5 min)
One sentence in your dev journal: what you built, what confused you, what you learned. After 9 months this journal is a record of your entire engineering education.

Weekend rhythm:

- **Saturday:** 4–5 hour Njirani sprint. Add a feature, not just fixes. Make visible progress.
- **Sunday morning:** Weekly review — did you hit your goal? Plan next week's focus. Push anything pending.
- **Sunday afternoon:** Full rest. Go outside. Your brain rebuilds during downtime.

8. Tools & Platforms

Tool	What you use it for
Notion	Weekly planner, reading notes, dev journal, feature kanban board. Set up: Backlog → In Progress → Done
Linear	Treat Njirani like a real product. Create issues, assign them to yourself, close them when done.
Obsidian	Local markdown notes for deep technical concepts. Link ideas together. Searchable forever.
Toggl Track	Free time tracker. Know exactly where your hours go. Weekly report shows the truth.
drawsql.app	Design your database schema visually before writing a single migration.
TablePlus	GUI client for PostgreSQL. Browse your data visually while you build.
Insomnia / Postman	Test your API endpoints manually during development.
Cloudflare Tunnel	Expose your localhost over public HTTPS. Essential for M-Pesa Daraja callbacks.

9. Books Worth Buying

In order of when to read them:

Month 2	Python Testing with pytest by Brian Okken The definitive pytest book. Covers everything from basics to advanced patterns. Read during your testing week.
Month 5	Designing Data-Intensive Applications by Martin Kleppmann The most important backend engineering book that exists. Start in Month 5, read slowly over 3 months. Changes how you think about systems.
Any month	The Pragmatic Programmer by Hunt & Thomas Read a chapter a week. Changes how you think about the craft of engineering, not just code.
Month 4	Clean Code by Robert Martin Controversial but important. Read it critically — take the useful parts, question the rest.

10. The One Metric That Matters

At the end of every month ask yourself one question: "Can I explain to a stranger, in plain English, what I built this month, why I built it, and what I learned?" If yes — you're on track. If no — you went too fast or spread too thin. Slow down and consolidate before moving forward.

What you'll have at the end of Month 9:

- A live product on your own domain with real users (or at least a real demo)
- M-Pesa Daraja integration working in production
- A tested, containerised, auto-deploying API
- A Next.js web client and a Flutter mobile client consuming the same API
- One AI-powered feature built with the Claude API
- Infrastructure defined entirely as code with Terraform
- A green GitHub profile with 9 months of consistent commits
- A technical blog with posts people can find on Google
- A portfolio that speaks louder than any certificate

That is not a student project. That is the portfolio of a backend engineer. You will have earned every line of it. Tukae kazi.

Enock Kipkorir · Multimedia University of Kenya · enockkipkorir594.github.io/Portfolio2.0